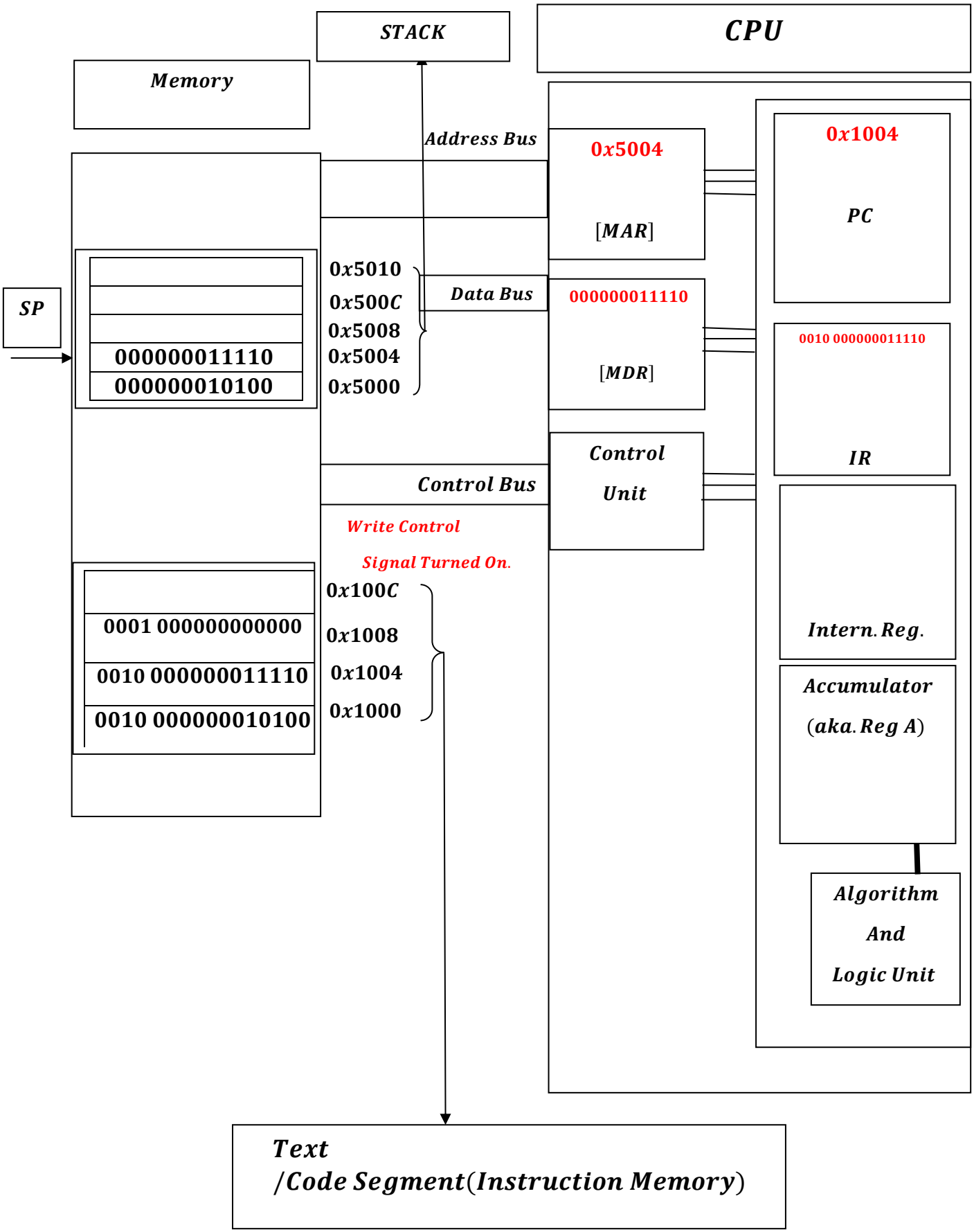
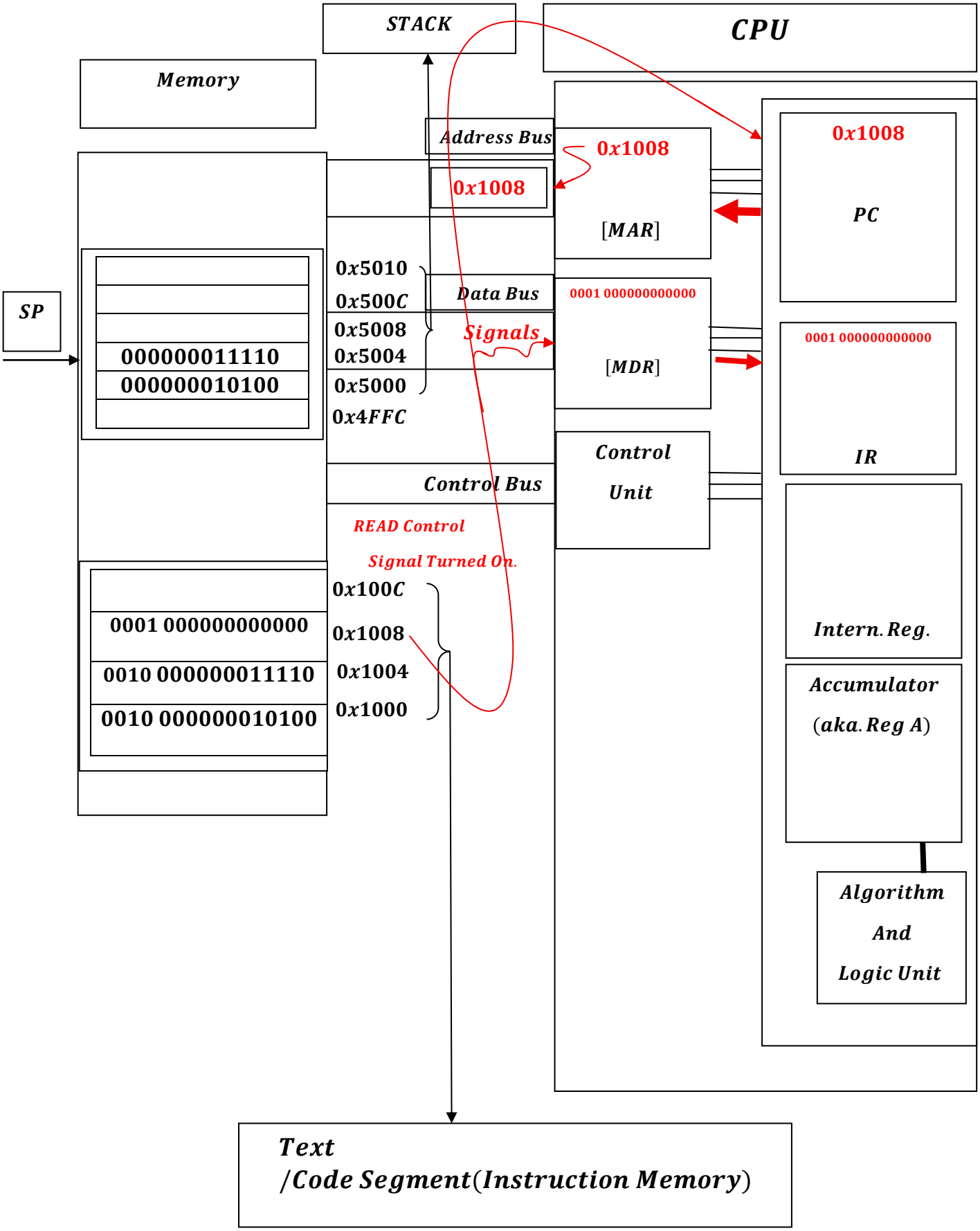
Operation Mechanism – Hardware Memory Mechanism



In the same way IR recieves, the instruction:



[*Instruction Register*] *Holds*: 0001 000000000000 <-- (*The compiler wrote this*)

| (*The CU reads* "0001" *and activates its hardwired* "*ADD Recipe*")

↓

[*CONTROL UNIT*]

|

|--- (*Action* 1: *Get first number*)

|-------> *Sends SP Address* (0x5004) *to MAR*

|-------> *Turns on READ signal on Control Bus*

|-------> *Routes Data* (30) *from MDR to Internal Reg*

|-------> *Sends electrical pulse to hardware subtractor*: $SP = SP - 4$

|

|--- (*Action* 2: *Get second number*)

|-------> *Sends SP Address* (0x5000) *to MAR*

|-------> *Turns on READ signal on Control Bus*

|-------> *Routes Data* (20) *from MDR to Accumulator*

|-------> *Sends electrical pulse to hardware subtractor*: $SP = SP - 4$

|

|--- (*Action* 3: *Do the Math* & *Push Result*)

|-------> *Triggers ALU to do* $20 + 30 = 50$

|-------> *Sends electrical pulse to hardware adder*: $SP = SP + 4$

|-------> *Sends Address to MAR*, 50 *to MDR, turns on WRITE signal*

1. *Routing the Address* (*Inside the CPU*)

Before anything else can happen, the Control Unit (*CU*) *looks at the Stack Pointer Register to see where the top of the stack currently is. In our scenario, the SP holds* 0x5004. *The CU takes that address and sends it directly to the Memory Address Register* (*MAR*). *The CU routes this Stack Address to the MAR directly using the CPU's internal wiring, absolutely not through the Address Bus.*

==========

$STEP\ 1: INTERNAL\ ADDRESS\ ROUTING$

==========

[$Stack\ Pointer\ (SP)$] $Holds: 0x5004$

$\downarrow$ $(CU\ routes\ Address\ via\ internal\ wires)$

[$Memory\ Addr.Reg\ (MAR)$] $Now\ Holds: 0x5004$

$2. The\ Memory\ Request\ (Reaching\ outside\ the\ CPU)$

$Now\ the\ CPU\ must\ talk\ to\ the\ main\ memory. The\ MAR\ places\ the\ address\ 0x5004\ onto\ the\ Address\ Bus, which\ is\ hardwired\ directly\ into\ the\ Memory\ Unit's\ input\ pins. At\ the\ exact\ same\ time, the\ Control\ Unit\ sends\ a\ high-voltage\ electrical\ signal\ (1) down\ the\ READ\ wire\ of\ the\ Control\ Bus.$

==========

$STEP\ 2: THE\ READ\ COMMAND$

==========

CPU $STACK\ MEMORY$

[MAR] $---$ ($Address\ Bus: 0x5004$) $--->$ [$Address\ Decoder$]

|

[CU] $---$ ($Control\ Bus: READ$) $--->$ [$Wake\ up$ & $Send!$]

3. *The Dispatch and the Catch (Data returns to CPU)*

Inside the physical Memory Unit, the Address Decoder reads 0x5004 from the Address Bus, navigates its vast grid of transistors, and physically connects to that specific row of memory cells. Because the READ signal is active, the Memory Unit takes the binary data stored there (the 30) and pushes those electrical signals onto the Data Bus.

The Data Bus carries this data straight back to the CPU. The CPU waits a tiny fraction of a microsecond, and then the Control Unit triggers the MDR to latch *or save whatever is currently on the Data Bus.*

```
==========================================
                STEP 3: THE DATA CATCH
==========================================
        CPU                                STACK MEMORY
                                           [ Address 0x5004 ]
[ MDR ] <---(Data Bus: 30) ---------- (Holds: 30)
```

4. *The Internal Routing (MDR → Internal Register)*

Now the data 30 is safely inside the CPU's loading dock (the MDR). The CU Routes *this data by opening an internal pathway. The data moves via internal CPU wiring directly from the MDR into the dedicated Internal Register.*

==========

$STEP\ 4: INTERNAL\ DATA\ ROUTING$

==========

[$Memory\ Data\ Reg\ (MDR)$] $Holds: 30$

$\downarrow$ $(CU\ opens\ logic\ gates\ for\ transfer)$

[$Internal\ Register$] $Now\ Holds: 30 \quad <-- \quad (Ready\ for\ the\ ALU!)$

$5.\ Decrementing\ the\ Stack\ Pointer$

$Now\ that\ the\ data\ has\ been\ successfully\ pulled\ off\ the\ top\ of\ the\ stack,$

$the\ CU\ sends\ a\ command\ to\ the\ Hardware\ Subtractor\ attached\ to\ the\ SP.$

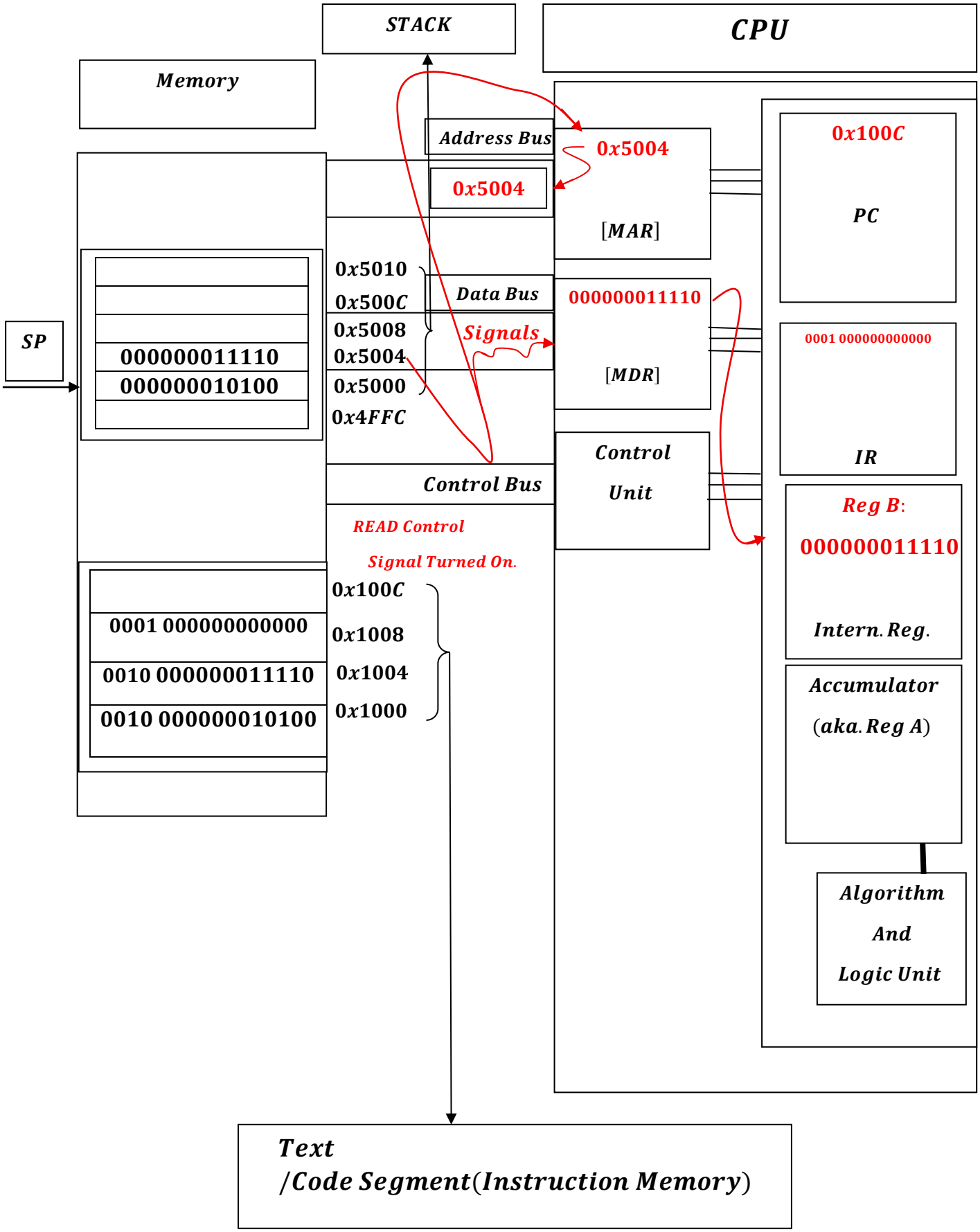
$The\ physical\ SP\ register\ is\ updated\ from\ 0x5004\ to\ 0x5000.$

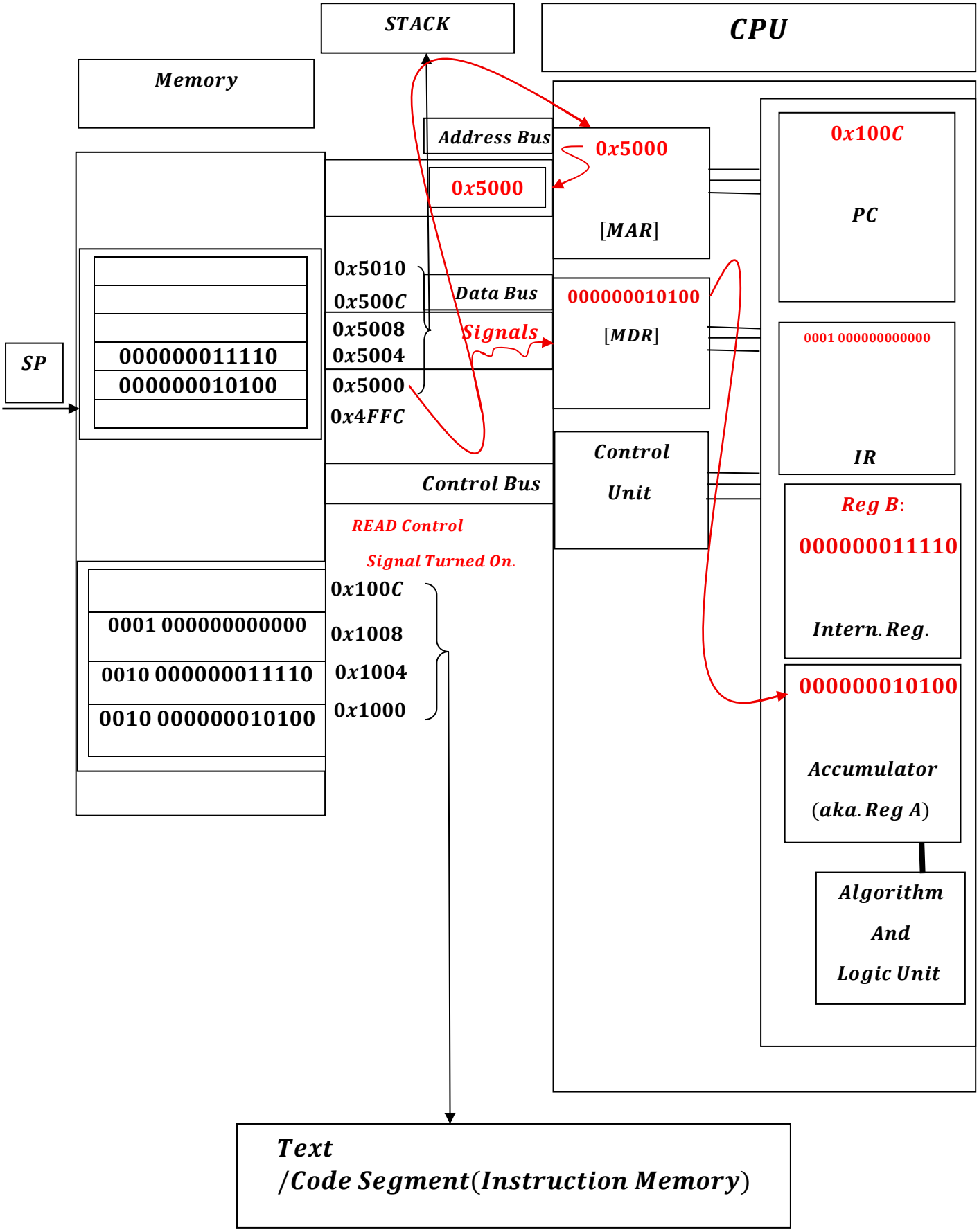
$\underline{Repeating\ for\ the\ Second\ Value\ (20)}$

$Once\ the\ 30\ is\ completely\ locked\ into\ the\ Internal\ Register\ and\ the\ SP\ has\ been\ decremented, the\ CU\ repeats\ the\ exact\ same\ 5-step\ process\ at\ lighting\ speed\ to\ get\ the\ second\ operand:$

1. $Address: SP\ (0x5000)\ is\ internally\ routed\ to\ MAR.$
2. $Request: MAR\ puts\ 0x5000\ on\ the\ Address\ Bus;\ CU\ turns\ on\ the\ READ\ signal.$
3. $Catch: The\ Memory\ puts\ 20\ on\ the\ Data\ Bus;\ MDR\ latches\ it.$
4. $Route: CU\ routes\ the\ data\ 20\ via\ internal\ wiring\ from\ the\ MDR\ straight\ into\ the\ Accumulator\ (Reg\ A).$
5. $Decrement: SP\ decrements\ to\ 0x4FFC.$

$Now\ both\ numbers\ are\ sitting\ perfectly\ in\ the\ exact\ registers\ connected\ to\ the\ Algorithm\ And\ Logic\ Unit, and\ the\ CPU\ is\ finally\ ready\ to\ execute\ the\ actual\ math!$


STACK
CPU
Memory
Address Bus
0x5004
0x5004
[MAR]
0x100C
PC
0x5010
0x500C
Data Bus
000000011110
0x5008
Signals
0x5004
000000011110
000000010100
0x5000
0x4FFC
[MDR]
0001 000000000000
IR
SP
Control Bus
Control Unit
READ Control
Signal Turned On.
Reg B:
000000011110
Intern. Reg.
0x100C
0001 000000000000
0x1008
0010 000000011110
0x1004
0010 000000010100
0x1000
Accumulator
(aka. Reg A)
Algorithm
And
Logic Unit
Text
/Code Segment(Instruction Memory)


STACK
CPU
Memory
Address Bus
0x5000
0x5000
[MAR]
0x100C
PC
0x5010
0x500C
0x5008
0x5004
0x5000
0x4FFC
Data Bus
Signals
000000010100
[MDR]
0001 000000000000
IR
SP
000000011110
000000010100
Control Unit
Control Bus
READ Control
Signal Turned On.
Reg B:
000000011110
Intern. Reg.
0x100C
0x1008
0x1004
0x1000
0001 000000000000
0010 000000011110
0010 000000010100
000000010100
Accumulator
(aka. Reg A)
Algorithm
And
Logic Unit
Text
/Code Segment(Instruction Memory)

$\boldsymbol{What\ PC(Program\ Counter)\ will\ do\ at\ this\ time}$?

$\boldsymbol{The\ "Fetch"\ Phase\ vs. The\ "Execution"\ Phase}$

$The\ MAR\ acts\ as\ the\ single\ doorway\ to\ the\ outside\ memory. Both\ the\ PC\ and\ the$

$SP\ have\ to\ take\ turns\ using\ the\ MAR\ depending\ on\ what\ phase\ the\ CPU\ is\ in.$

$\mathbf{1}. \boldsymbol{The\ Fetch\ Phase\ (PC's\ Turn)}$

$When\ the\ CPU\ needs\ to\ know\ what\ instruction\ to\ do\ next, it\ is\ the\ PC's\ time\ to\ shine.$

- $The\ PC\ holds\ 0x1008\ (the\ address\ of\ the\ ADD\ command).$
- $The\ PC\ sends\ 0x1008\ to\ the\ MAR.$
- $The\ CPU\ fetches\ the\ ADD\ opcode\ (0001\ 000000000000)\ from\ the\ Text/$
  $Code\ Segment\ and\ puts\ it\ in\ the\ Instruction\ Register\ (IR).$
- $\boldsymbol{Crucial\ Step}: The\ PC\ immediately\ uses\ its\ hardwired\ PC\ Incrementer\ to\ add\ +$
  $4. The\ PC\ now\ holds\ 0x100C.$
- $The\ PC\ is\ now\ done. It\ goes\ to\ sleep\ and\ waits\ for\ the\ next\ cycle.$

$\mathbf{2}. \boldsymbol{The\ Execution\ Phase\ (SP's\ Turn)}$

$Now\ the\ instruction\ 0001\ (ADD) is\ sitting\ in\ the\ Control\ Unit\ (CU). The\ CU\ starts\ its$

ADD Recipe. $Because\ this\ recipe\ requires\ grabbing\ data\ from\ the\ stack,$

$it\ completely\ ignores\ the\ PC. It\ is\ now\ the\ SP's\ turn\ to\ use\ the\ MAR.$

Execution Phase

Step 1: The Execution (Inside the ALU)

Right now, the 20 is sitting in the Accumulator (also known as Reg A). The 30 is sitting in the Internal Register. The Control Unit sends the signal to the Algorithm And Logic Unit to execute the addition.

==========

STEP 1: THE ALU ADDITION

==========

[*Internal Register*] *Holds*: 30

↓

[*ALGORITHM AND LOGIC UNIT*] <--- *Performs*: 20 + 30

↑

[*Accumulator (Reg A)*] *Holds*: 20

The ALU processes the binary addition at lightning speed. It outputs the result (50) and instantly overwrites the old data in the Accumulator. Now, the Accumulator safely holds the final answer: 50.

Step 2: Preparing the Stack Address (Incrementing the SP)

Before the CPU can push this new data out to memory, it needs to find an empty slot. Because we popped two numbers, the stack is temporarily empty, and the Stack Pointer (SP) is sitting at 0x4FFC.

The Control Unit acting as a Manager orders the Stack Pointer to calculate the next Address. The Hardware Adder attached to the SP receives the electrical command signal (ADD 4)***and automatically calculates***: $0x4FFC + 4 = 0x5000$ ***. This completely overwrites the SP with the new*** $0x5000$.

Now that the SP is ready, the Control Unit routes the Stack Address directly to the Memory Address Register (***MAR***).

==========

STEP 2: ***ADDRESS ROUTING***

==========

[***Stack Pointer*** (***SP***)]

Calculates: $0x4FFC + 4 = 0x5000$

↓ (***CU routes Address via internal wires***)

[***Memory Addr. Reg*** (***MAR***)] ***Now Holds***: $0x5000$

Step 3: ***Routing the Result to the Outbound Loading Dock***

Because all data leaving the CPU must go through the Memory Data Register (***MDR***), ***the Control Unit opens an internal pathway. It routes the operand*** (***the result*** 50) ***from the Accumulator straight into the MDR***.

==========

STEP 3: DATA ROUTING

==========

[***Accumulator (Reg A)***] ***Holds***: **50**

| (***CU routes Data via internal wires***)

↓

[***Memory Data Reg (MDR)***] ***Now Holds***: **50**

Step 4: The WRITE Command (The Final Push)

Everything is staged and ready. The MAR holds the destination, and the MDR holds the payload. Now the CU pulls the trigger to push the data out to the STACK Memory.

1. ***Address Bus: The MAR places the address 0x5000 onto the Address Bus.***
2. ***Data Bus: The MDR pushes the binary data (50) out onto the Data Bus.***
3. ***Control Bus: The Control Unit sends the WRITE signal to the Control Bus, turning it on.***

==========

STEP 4: THE WRITE EXECUTION

==========

CPU ***STACK MEMORY***

[***MAR***] --- (***Address Bus***: **0x5000**) ---> [***Address Decoder***]

|

[***CU***] --- (***Control Bus***: ***WRITE***) ---> [***Open the gates!***]

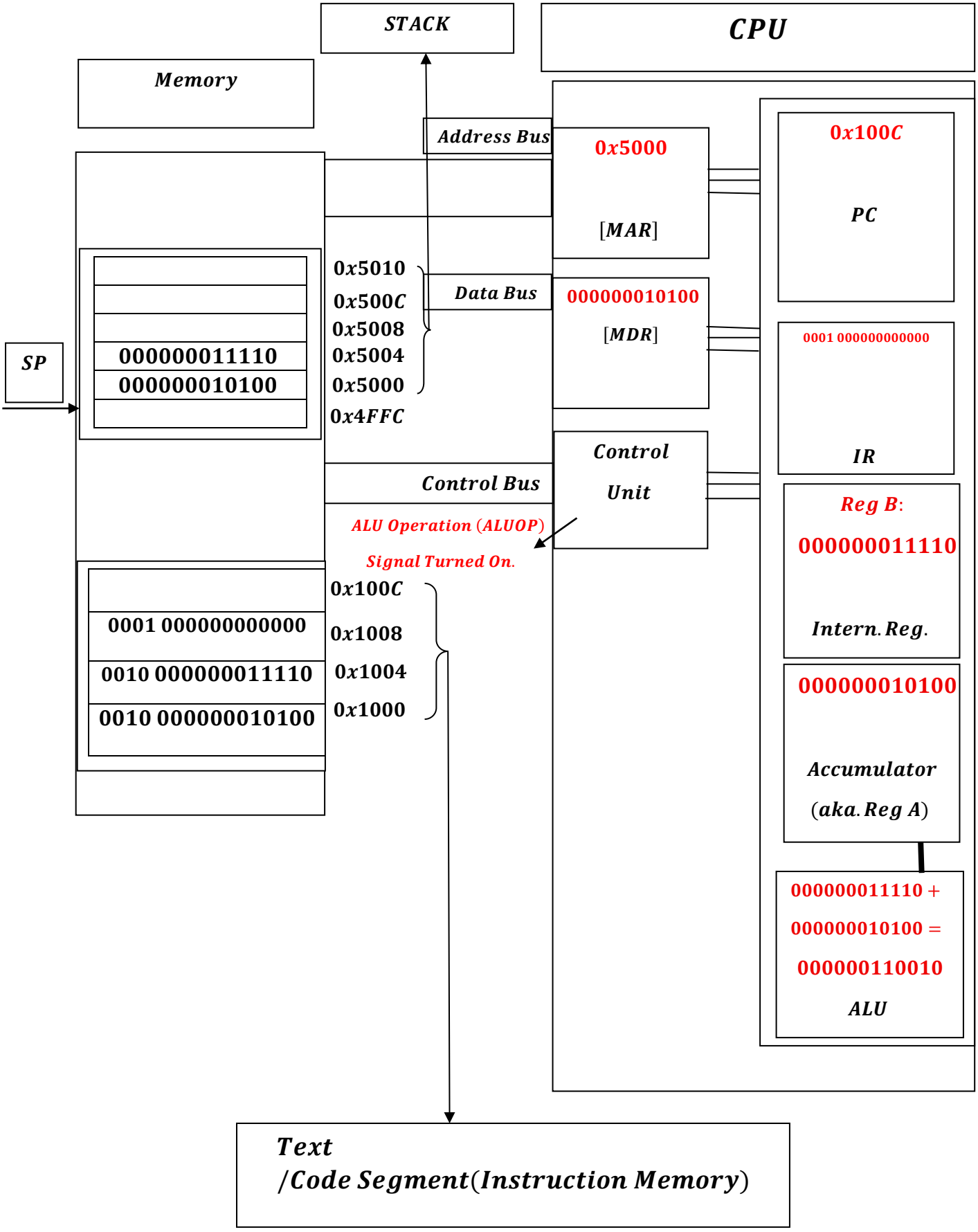
|

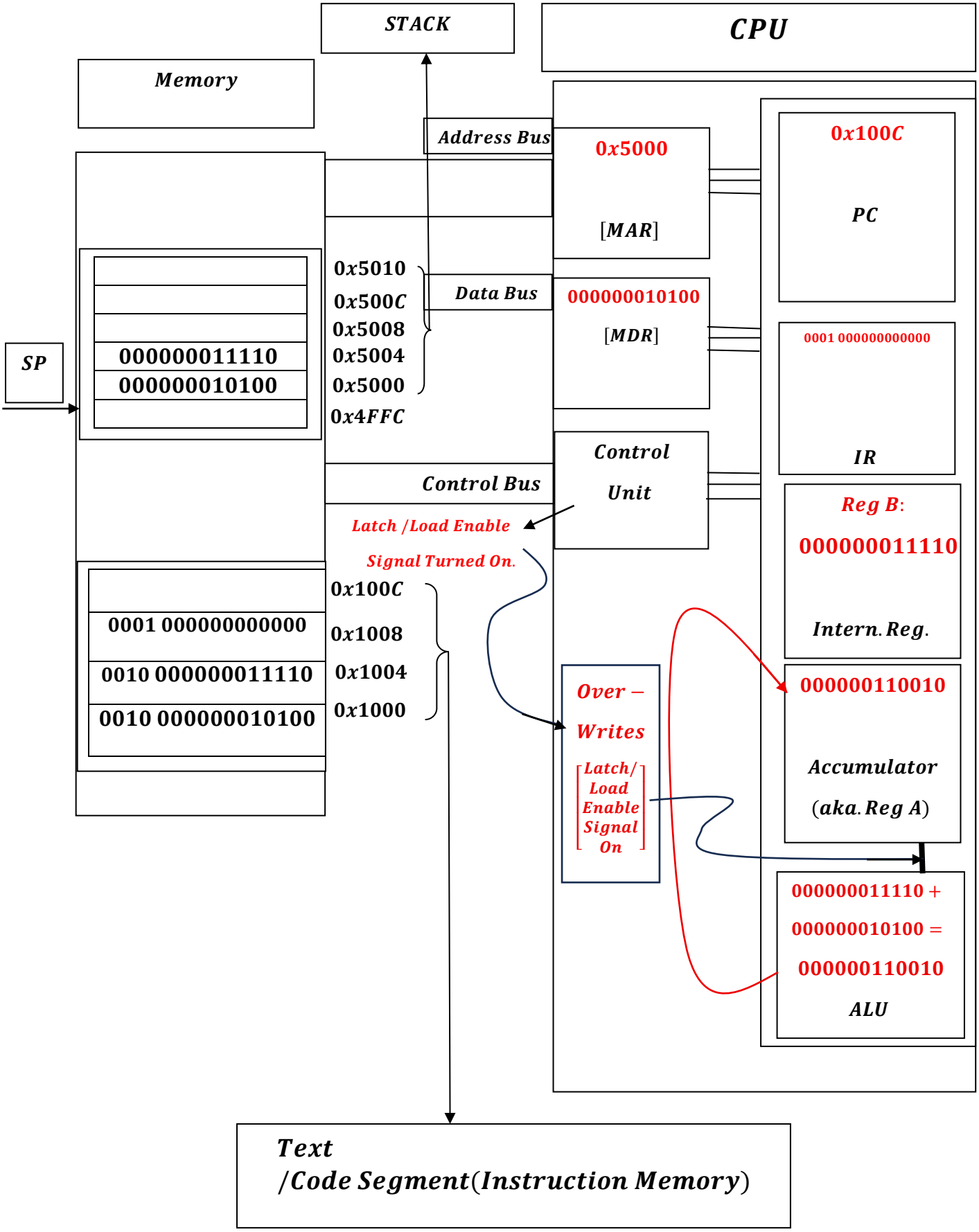
[***MDR***] --- (***Data Bus***: **50**) ---> [***Overwrite*** **0x5000**]

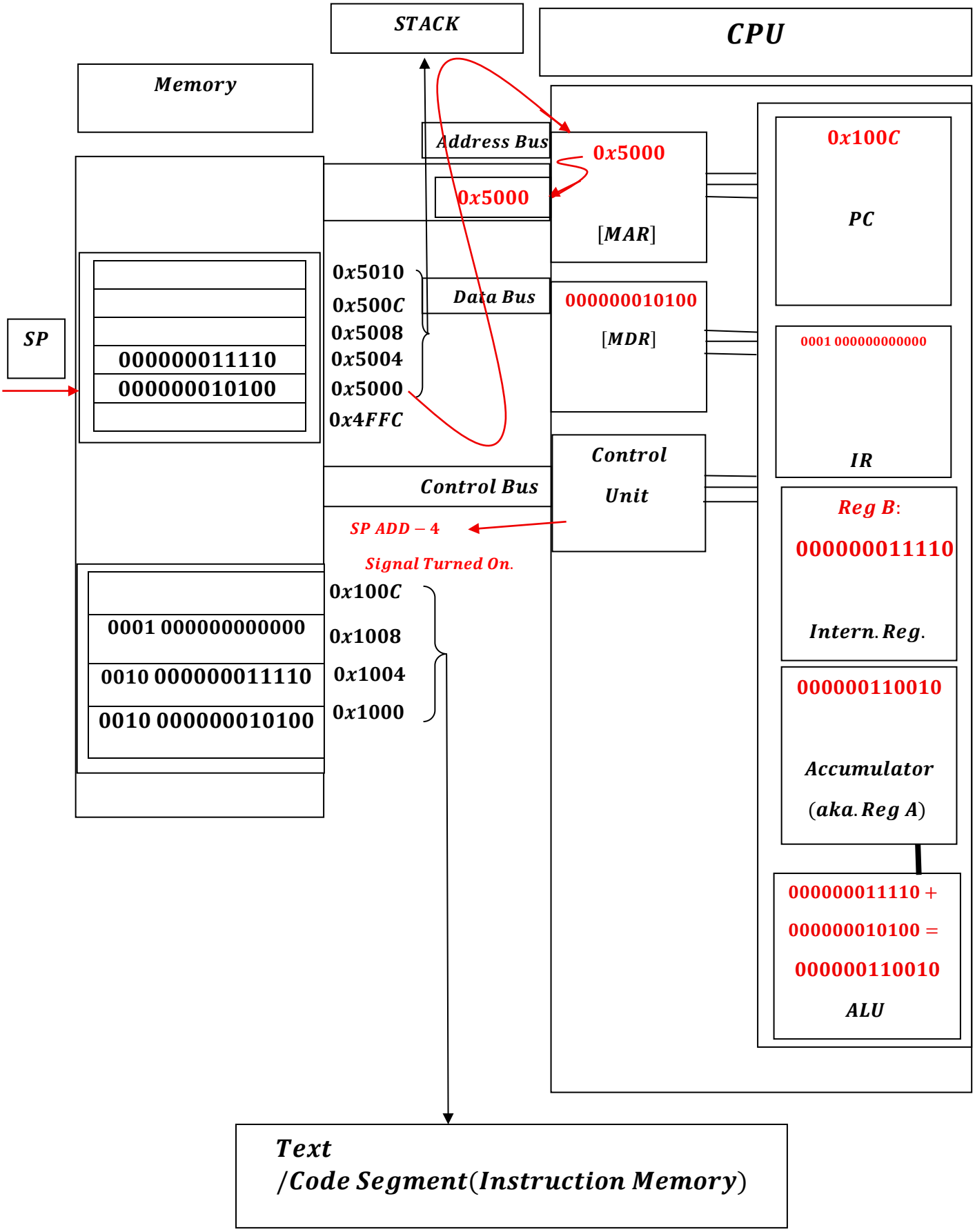
$When\ the\ Memory\ Unit\ receives\ the\ high - voltage\ WRITE\ signal,$
$it\ looks\ at\ the\ Address\ Bus\ to\ find\ slot\ 0x5000.\ It\ looks\ at\ the\ Data\ Bus,$
$grabs\ the\ 50,\ and\ physically\ latches\ that\ electrical\ state\ into\ the\ transistors$
$at\ that\ exact\ location.$

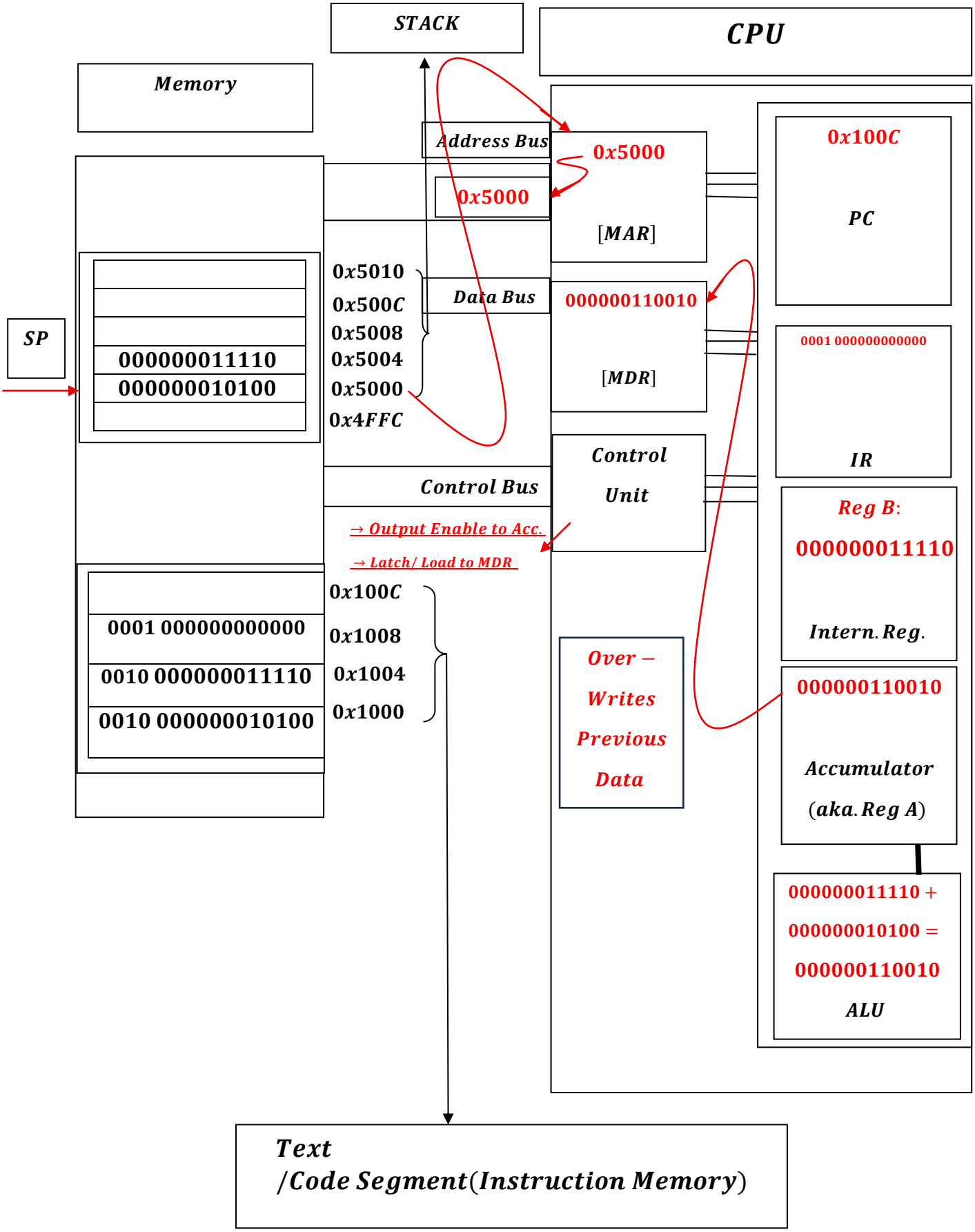
$The\ old\ data\ (20)\ that\ was\ previously\ sitting\ at\ 0x5000\ is\ utterly\ destroyed$
$and\ overwritten\ by\ the\ new\ sum.$

$The\ ADD\ instruction\ is\ now\ officially\ 100\%\ complete!\ The\ CPU\ is\ done.$


STACK
CPU
Memory
Address Bus
0x5000
[MAR]
0x100C
PC
0x5010
0x500C
0x5008
0x5004
0x5000
0x4FFC
Data Bus
000000010100
[MDR]
0001 000000000000
IR
SP
000000011110
000000010100
Control Unit
Control Bus
ALU Operation (ALUOP)
Signal Turned On.
Reg B:
000000011110
Intern. Reg.
0x100C
0001 000000000000
0x1008
0010 000000011110
0x1004
0010 000000010100
0x1000
000000010100
Accumulator
(aka. Reg A)
000000011110 +
000000010100 =
000000110010
ALU
Text
/Code Segment(Instruction Memory)


STACK
CPU
Memory
Address Bus
0x5000
[MAR]
0x100C
PC
0x5010
0x500C
0x5008
0x5004
0x5000
0x4FFC
Data Bus
000000010100
[MDR]
0001 000000000000
IR
SP
000000011110
000000010100
Control Unit
Control Bus
Latch /Load Enable
Signal Turned On.
Reg B:
000000011110
Intern. Reg.
0x100C
0x1008
0x1004
0x1000
0001 000000000000
0010 000000011110
0010 000000010100
Over – Writes
[Latch/ Load Enable Signal On]
000000110010
Accumulator
(aka. Reg A)
000000011110 +
000000010100 =
000000110010
ALU
Text
/Code Segment(Instruction Memory)


STACK
CPU
Memory
Address Bus
0x5000
0x5000
[MAR]
0x100C
PC
0x5010
0x500C
0x5008
0x5004
0x5000
0x4FFC
Data Bus
000000010100
[MDR]
0001 000000000000
IR
SP
000000011110
000000010100
Control Unit
Control Bus
SP ADD – 4
Signal Turned On.
Reg B:
000000011110
Intern. Reg.
0x100C
0001 000000000000
0x1008
0010 000000011110
0x1004
0010 000000010100
0x1000
000000110010
Accumulator
(aka. Reg A)
000000011110 +
000000010100 =
000000110010
ALU
Text
/Code Segment(Instruction Memory)


STACK
CPU
Memory
Address Bus
0x5000
0x5000
[MAR]
0x100C
PC
SP
000000011110
000000010100
0x5010
0x500C
0x5008
0x5004
0x5000
0x4FFC
Data Bus
000000110010
[MDR]
0001 000000000000
IR
Control Unit
Control Bus
→ Output Enable to Acc.
→ Latch/ Load to MDR
Reg B:
000000011110
Intern. Reg.
0001 000000000000
0010 000000011110
0010 000000010100
0x100C
0x1008
0x1004
0x1000
Over – Writes Previous Data
000000110010
Accumulator
(aka. Reg A)
000000011110 +
000000010100 =
000000110010
ALU
Text
/Code Segment(Instruction Memory)

==========

THE PRECISE INTERNAL ROUTING (***Accumulator*** −> ***MDR***)

==========

[***CONTROL UNIT***]

|

| − − − (***Signal*** **1**: "***Output Enable***")

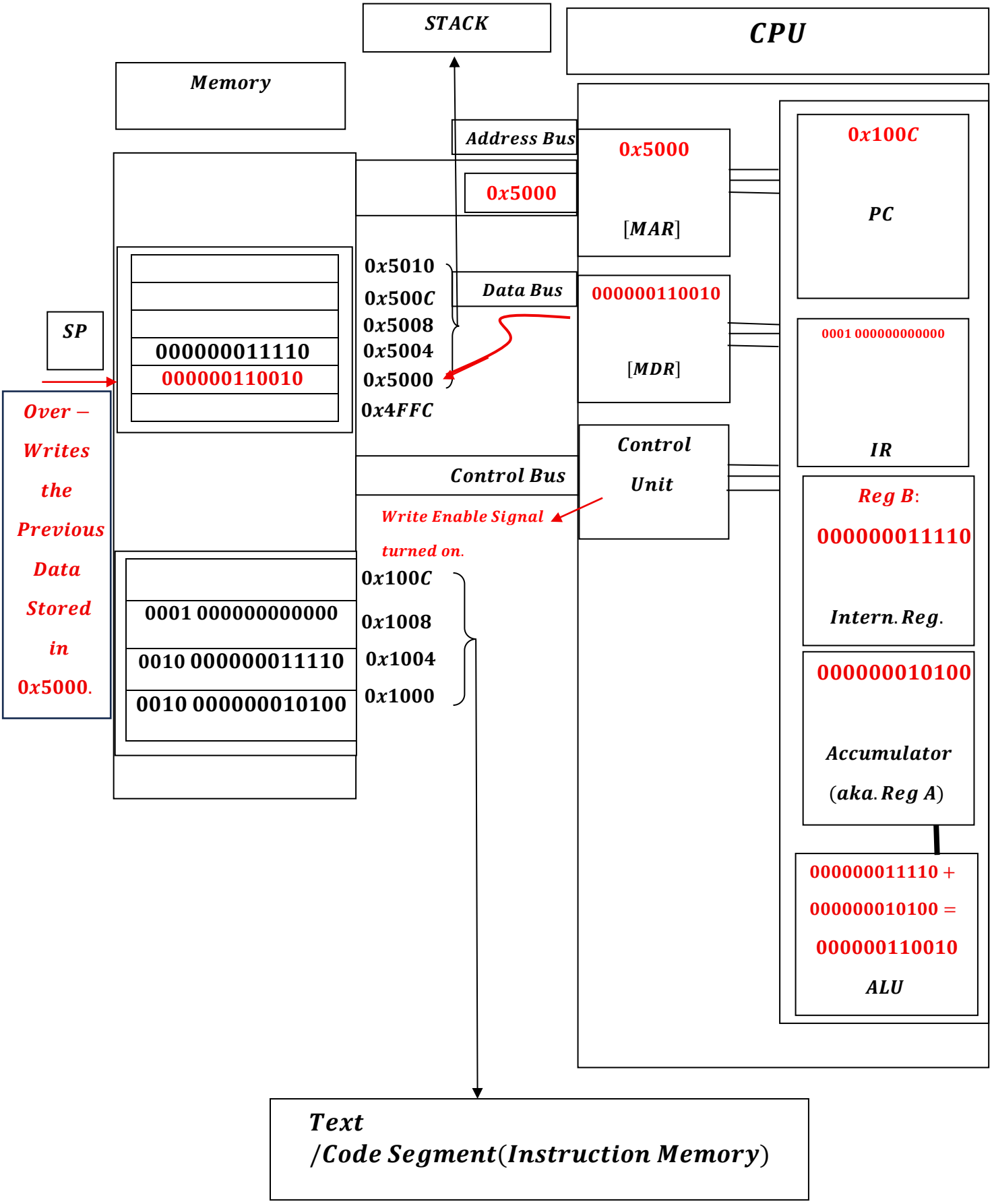
| − − − − − − −> ***Sent to*** [***Accumulator***] : "***Release the*** **50** ***internally***! "

|

| − − − (***Signal*** **2**: "***Load / Latch***")

| − − − − − − −> ***Sent to*** [***MDR***] : "***Catch the*** **50**! "

− − − − − − −


STACK
CPU
Memory
Address Bus
0x5000
0x5000
[MAR]
0x100C
PC
0x5010
0x500C
0x5008
0x5004
0x5000
0x4FFC
Data Bus
000000110010
[MDR]
0001 000000000000
IR
SP
000000011110
000000110010
Over – Writes the Previous Data Stored in 0x5000.
Control Unit
Control Bus
Write Enable Signal turned on.
Reg B:
000000011110
Intern. Reg.
0x100C
0001 000000000000
0x1008
0010 000000011110
0x1004
0010 000000010100
0x1000
000000010100
Accumulator (aka. Reg A)
000000011110 + 000000010100 = 000000110010
ALU
Text /Code Segment(Instruction Memory)

Note , in mordern CPUs there are numerous internal registers , where data can be stored hence ACCUMULATOR will be used as REG A is not a hard − bound RULE.

Yes , Accumulator can be used as a storage purpose to store the resultant Data send by ALU .